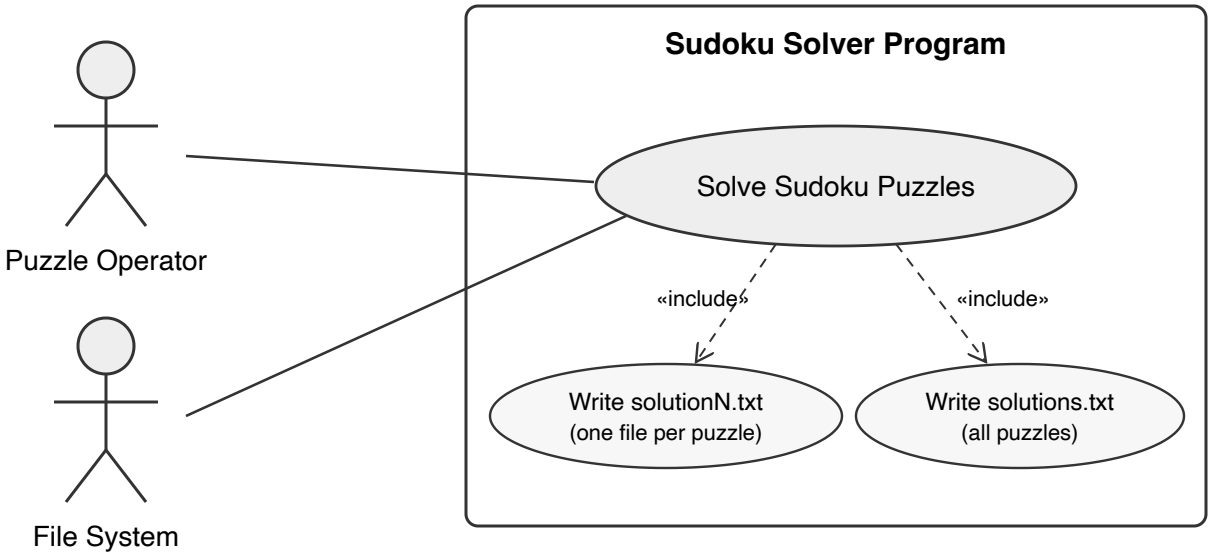
EECS 348 Assignment 6: UML Use Case

Francis Tryban · November 13, 2026 · Updated from Luke Tidball's Assignment 5 use case with Claude Opus 5.5 (through Claude Code).

a. Identify/discover a use case

- **Use case:** Solve Sudoku Puzzles
- **Primary actor:** Puzzle Operator (the person running the program)
- **Supporting actor:** File System (gives the puzzle files, saves the output files)
- **Goal:** Solve every puzzle and save the answers in solutionN.txt and in one solutions.txt.
- **Use case diagram:**



b. Brief description

The operator runs the program. It reads each puzzle, solves it, and saves the answers two ways: in that puzzle's own solutionN.txt (from Assignment 5) and in one shared solutions.txt (new in Assignment 6). In solutions.txt, each puzzle starts with "Puzzle: <file name>".

c. Step-by-step flow (outline)

1. Operator starts the program.
2. Program makes solutions.txt.
3. Program picks the next puzzle file.
4. Program writes "Puzzle: <file name>" to solutions.txt.
5. Program reads the puzzle file.
6. Program checks the puzzle is valid.
7. Program solves it.
8. Program writes the answers to solutionN.txt.
9. Program writes the same answers to solutions.txt.

10. Repeat steps 3–9 for each puzzle.
11. Program ends.

d. Detailed description

Use case: Solve Sudoku Puzzles

Actors: Puzzle Operator (primary), File System (supporting).

Preconditions

- The operator can run the program.
- The puzzle files are in the same folder.
- The program can make files in that folder.

Basic flow

1. **Start.** The operator runs the program. If they type file names, it uses those. If not, it uses puzzle1–5.txt.
2. **Make solutions.txt.** The program makes solutions.txt once at the start, so every puzzle goes in the same file.
3. **Pick a puzzle.** The program takes the next file and writes "Puzzle: <file name>" to solutions.txt. It also opens that puzzle's solutionN.txt.
4. **Read it.** The program reads the puzzle from the file.
5. **Check the format.** It needs nine rows of nine cells. Each cell is 1–9 or _ for blank.
6. **Check the starting numbers.** No repeats in any row, column or 3×3 box.
7. **Solve.** The program tries numbers and backs up when it gets stuck, until it finds every answer. It only solves each puzzle once.
8. **Write to solutionN.txt.** Every answer, then "Solutions found: n". If none, "No solution found."
9. **Write to solutions.txt.** The same answers, under that puzzle's "Puzzle:" line.
10. **Next puzzle.** Repeat steps 3–9.
11. **Done.** The program closes the files and exits.

Alternative courses of action

- **A1. Can't open a puzzle file.** Prints an error, writes "No solution found." in solutions.txt, and moves on.
- **A2. Puzzle file is formatted wrong.** Same as A1.
- **A3. Starting numbers break a rule.** Writes "No solution found." to both files and moves on.
- **A4. No answer exists.** Writes "No solution found." to both files and moves on.
- **A5. More than one answer.** Writes all of them and the count to both files.
- **A6. Can't make solutions.txt.** Prints an error and stops.
- **A7. Can't make a solutionN.txt.** Prints an error, writes "No solution found." in solutions.txt, and moves on.

Postconditions

- solutions.txt has a "Puzzle:" section for every file, in order.
- Every puzzle that could be read has its own solutionN.txt.